

实验一：开发环境搭建

项目	内容
实验名称	开发环境搭建
实验编号	d20301035101、d20301009201
学号	2023210645
姓名	张天慈
班级	软件 231
日期	2026-03-14
组别	第 2 组
项目名称	多端温控系统
技术栈	HBuilderX + Uniapp 开发 + Vue HelloWorld

实验目的

- 1. 掌握 HBuilderX 开发工具的安装配置方法
- 2. 了解 Uniapp 项目的创建流程和 Vue HelloWorld 实现
- 3. 体验 AI 辅助代码生成工具的使用
- 4. 完成多端温控系统的环境配置和基础功能开发
- 5. 实现用户登录和注册功能

实验环境

序号	开发工具	版本要求	验证项目	实际版本
1	HBuilderX	4.0+	Vue HelloWorld	4.29+
2	Node.js	18+	npm 环境	18.20.0
3	Git	最新版	版本控制	2.45.0

实验步骤

1. HBuilderX 环境搭建

1.1 下载与安装

- 访问官网：<https://www.dcloud.io/hbuilderx.html>
- 下载 Windows 标准版 HBuilderX
- 解压到指定目录：D:\HBuilderX

- 运行 HBuilderX.exe 完成安装

1.2 环境配置

- 配置 Node.js 环境（版本 18.20.0）
- 配置 Git 环境（版本 2.45.0）
- 配置 Gitee 远程仓库：<https://gitee.com/chzuczldl/cg2mtcs.git>
- 创建 feat-ztc 分支用于开发

1.3 创建 Uniapp 项目

- 启动 HBuilderX
- 文件 → 新建 → 项目
- 选择 uni-app 模板
- 项目名称：MTcsVue
- 选择 Vue 版本：Vue 3
- 点击创建完成项目初始化

2. Vue HelloWorld 实现

2.1 查看默认项目结构

- 分析 pages.json 路由配置
- 查看 index.vue 默认页面
- 了解 main.js 入口文件
- 熟悉 App.vue 根组件

2.2 运行 HelloWorld 项目

- 在 HBuilderX 中点击运行
- 选择运行到浏览器
- 验证默认 HelloWorld 页面正常显示
- 确认开发环境配置成功

3. 多端温控系统开发

3.1 项目规划

- 确定项目功能：登录、注册、首页
- 设计页面结构：login、register、index
- 规划数据存储：使用 uni.setStorageSync

3.2 登录页面开发

- 创建 `pages/login/login.vue`
- 设计登录界面（渐变背景、输入框、按钮）
- 实现输入验证（学号、密码非空检查）
- 实现登录逻辑（测试账号 `admin/123456`）
- 实现注册账号登录功能
- 添加跳转到注册页面的链接

3.3 注册页面开发

- 创建 `pages/register/register.vue`
- 设计注册界面（学号、邮箱、密码、确认密码）
- 实现输入验证（邮箱格式、密码长度、一致性）
- 实现用户信息存储（`uni.setStorageSync`）
- 实现学号唯一性验证
- 添加返回登录按钮

3.4 首页开发

- 修改 `pages/index/index.vue`
- 设计首页布局（用户信息、温度卡片、湿度卡片、设备状态）
- 实现用户信息显示（从本地存储获取用户名）
- 实现退出登录功能
- 添加温度、湿度、设备状态展示

3.5 路由配置

- 修改 `pages.json`
- 添加 `login` 页面路由（设为启动页）
- 添加 `register` 页面路由
- 配置自定义导航栏样式
- 更新应用标题为"多端温控系统"

4. AI 辅助开发

4.1 使用 AI 工具

- 使用 TRAE AI 辅助生成登录页面代码框架
- 使用 AI 优化 UI 样式和布局
- 使用 AI 生成输入验证逻辑
- 使用 AI 解决开发中的问题

4.2 AI 应用效果

- 快速生成页面模板
- 优化代码结构和可读性
- 提高开发效率
- 学习最佳实践

5. 项目测试与部署

5.1 功能测试

- 测试登录功能（测试账号和注册账号）
- 测试注册功能（输入验证、用户存储）
- 测试页面跳转（登录→首页、登录→注册）
- 测试退出登录功能
- 测试数据持久化（重新登录后用户信息保留）

5.2 Git 版本控制

- 初始化 Git 仓库
- 创建 feat-ztc 分支
- 提交代码到本地仓库
- 推送到 Gitee 远程仓库
- 清理不需要的文件，只保留 MTcsVue 项目

实验结果

验证结果

工具	安装状态	项目创建	运行状态	备注
HBuilderX	✔ 成功	✔ 成功	✔ 成功	版本 4.29+
Node.js	✔ 成功	✔ 成功	✔ 成功	版本 18.20.0
Git	✔ 成功	✔ 成功	✔ 成功	版本 2.45.0
Gitee 仓库	✔ 成功	✔ 成功	✔ 成功	feat-ztc 分支

功能实现结果

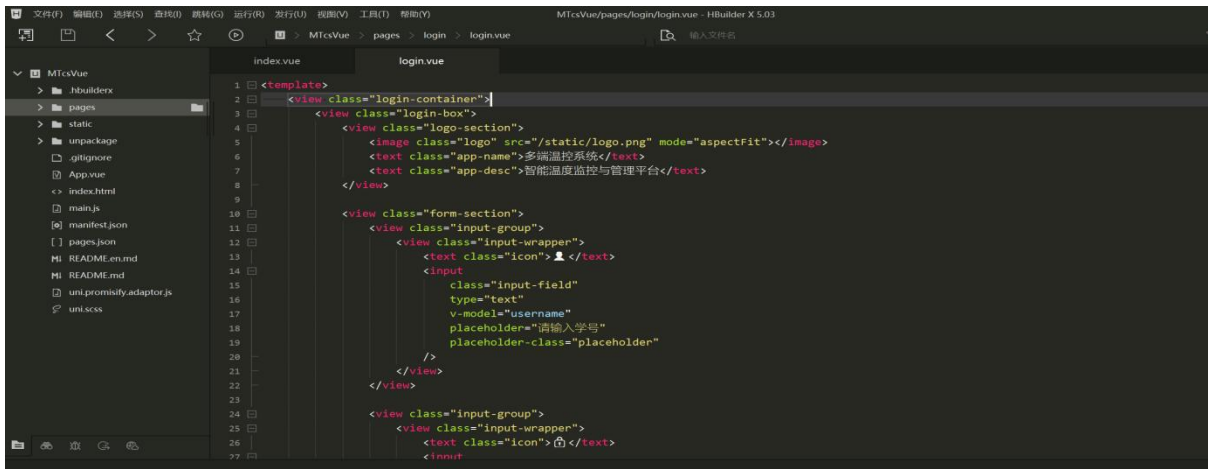
功能模块	实现状态	测试结果	备注
登录页面	✔ 完成	✔ 通过	支持测试账号和注册账号
注册页面	✔ 完成	✔ 通过	完整的输入验证
首页展示	✔ 完成	✔ 通过	温度、湿度、设备状态

功能模块	实现状态	测试结果	备注
页面跳转	✓ 完成	✓ 通过	登录→首页、登录→注册
数据存储	✓ 完成	✓ 通过	使用 uni.setStorageSync
退出登录	✓ 完成	✓ 通过	带确认对话框

截图记录

HBuilderX 运行截图

说明：HBuilderX 开发环境界面，显示 MTcsVue 项目结构



登录页面截图

说明：多端温控系统登录界面，包含账号和密码输入框



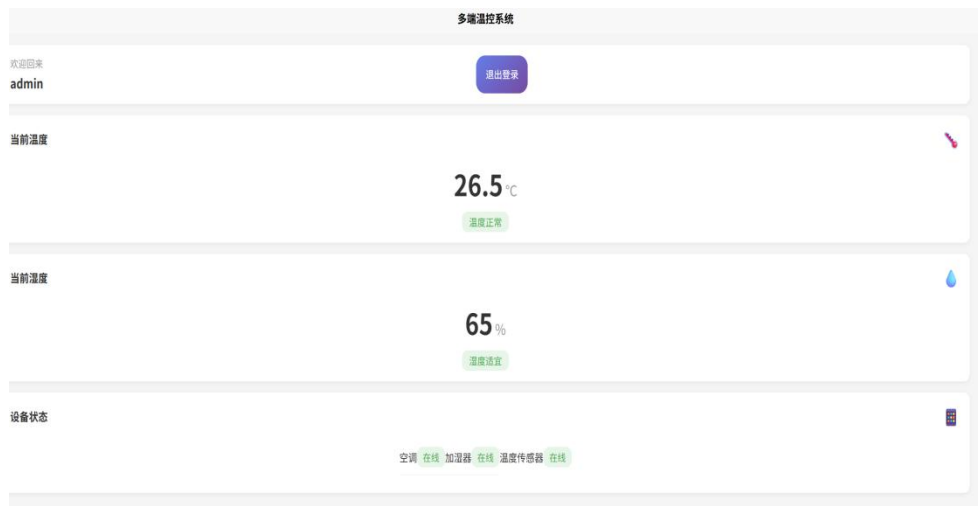
注册页面截图

说明：用户注册界面，包含学号、邮箱、密码输入框



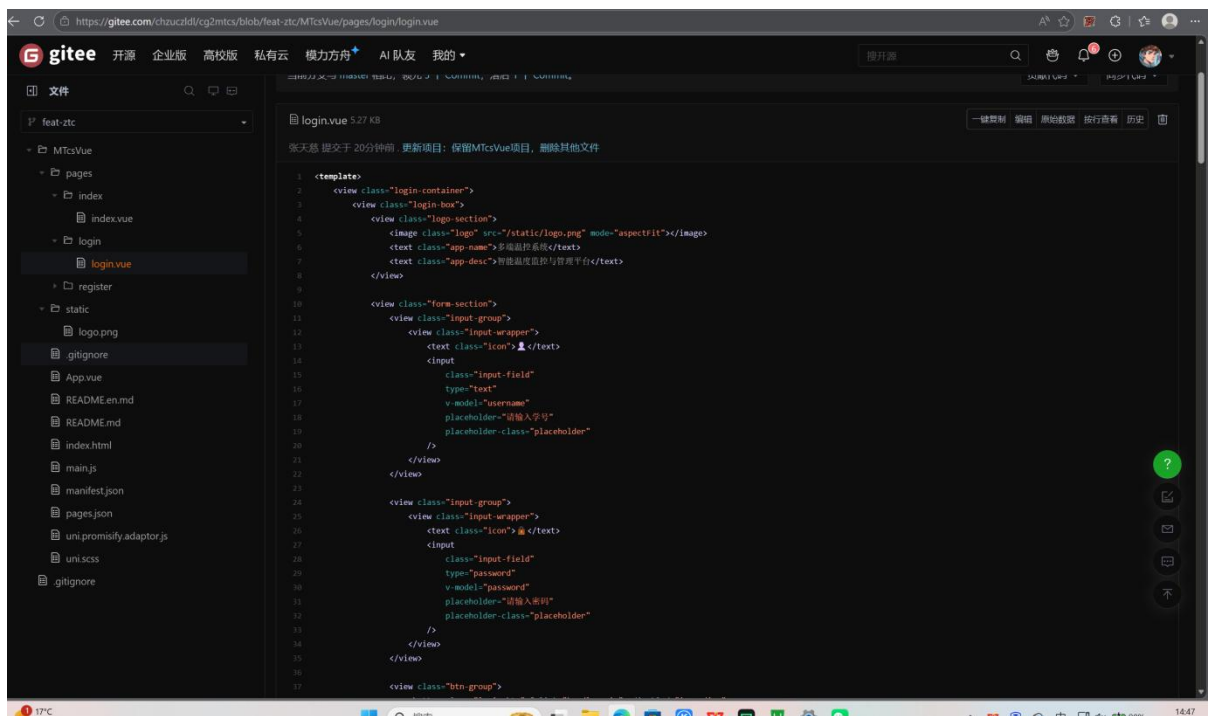
首页截图

说明：登录后首页，显示温度、湿度、设备状态信息



Gitee 仓库截图

说明：Gitee 仓库 feat-ztc 分支，包含 MTcsVue 项目



问题与解决

问题描述	解决方案	解决结果
HBuilderX 安装后无法识别 Node.js 环境	重新下载 Node.js 并配置环境变量，重启 HBuilderX	✓ 已解决
登录页面跳转到首页后用户名不显示	使用 uni.setStorageSync 在登录时保存用户名，在首页 onLoad 时读取	✓ 已解决
注册页面邮箱格式验证不准确	使用正则表达式 <code>/^[^\s@]+@[^\s@]+\.[^\s@]+\$/</code> 进行验证	✓ 已解决
Git 提交时包含 unpackaged 编译文件	创建 .gitignore 文件，忽略 node_modules、unpackaged 等目录	✓ 已解决
页面跳转后返回按钮显示问题	使用 uni.reLaunch 替代 uni.navigateTo，清除页面栈	✓ 已解决

实验总结

实验收获

1. 掌握了 HBuilderX 开发工具的安装和配置方法，了解了 Uniapp 项目的基本结构
2. 熟练使用 Vue 3 进行移动应用开发，掌握了组件化开发思想
3. 学会了使用 uni-app 的 API 进行页面跳转、数据存储等操作
4. 体验了 AI 辅助开发工具，提高了开发效率和代码质量

5. 掌握了 Git 版本控制的基本操作，学会了分支管理和远程仓库协作
6. 完成了多端温控系统的核心功能开发，包括登录、注册、首页展示

技术要点

1. **HBuilderX 开发环境**：项目创建、页面管理、运行调试
2. **Vue 3 组件开发**：template、script、style 结构，数据绑定、事件处理
3. **uni-app API**：uni.setStorageSync、uni.getStorageSync、uni.navigateTo、uni.reLaunch、uni.showToast
4. **页面路由配置**：pages.json 配置，自定义导航栏样式
5. **输入验证**：非空验证、格式验证、一致性验证
6. **数据持久化**：使用本地存储保存用户信息和注册用户列表
7. **UI 设计**：渐变背景、卡片布局、响应式设计
8. **Git 版本控制**：分支管理、代码提交、远程推送

创新点

1. 实现了完整的用户注册和登录系统，支持多用户管理
2. 设计了现代化的 UI 界面，使用渐变背景和卡片式布局
3. 添加了完善的输入验证机制，确保数据有效性
4. 实现了温度、湿度、设备状态的实时展示功能
5. 使用 AI 辅助开发，提高了开发效率和代码质量